DAIM SHORT COURSE · INTRODUCTION TO MACHINE LEARNING · WEEK 1

Understanding Machine Learning

How a computer learns from examples, and how we know whether it really has

Prof. Adil Khan

Centre of Excellence for Data Science, AI and Modelling (DAIM), University of Hull

> **How to use this reading**
>
> This is the pre-reading for the first live session. It tells the whole Week 1 story on its own: you do not need the slides, any code, or any maths beyond reading a chart. Set aside about forty minutes.
>
> Three small examples run through the whole document: filtering **spam emails**, estimating a **house price** or a **salary**, and predicting whether a **customer will buy**. Each time a new idea appears, it is attached to one of these, so by the end you will have seen the complete journey from "here is a problem" to "here is a model, and here is how far we can trust it".
>
> Shaded boxes like this one are worth pausing on. The questions at the end are the ones we will discuss in the webinar, so try them before you arrive.

The story in one paragraph. We will start with a task that is easy to describe but surprisingly hard to write rules for. We will see how giving a computer *examples* instead of *rules* gets around that difficulty, and that this is what "learning" means. We will then look closely at what those examples look like as data, meet the two main kinds of learning task, and open up the words "model" and "fitting" to see what is really inside them. Finally we arrive at the question that matters most in practice, and that this course keeps returning to: how do we know whether a model works on cases it has never seen?

1. Where machine learning sits

"Artificial intelligence" is used for almost anything a computer does that seems clever, so it helps to draw the map before we set off:

- **Artificial intelligence (AI)** is the broad field of building systems that perform tasks we associate with intelligence: recognising a face, translating a sentence, planning a route. There are many ways to build such systems. Some rely on rules written out by people, as a chess program of the 1990s did.
- **Machine learning (ML)** is the approach that instead lets the system work out its own patterns from data. It is the engine behind nearly everything called AI today, and it is what this course is about. Inside ML you will hear a few more names.

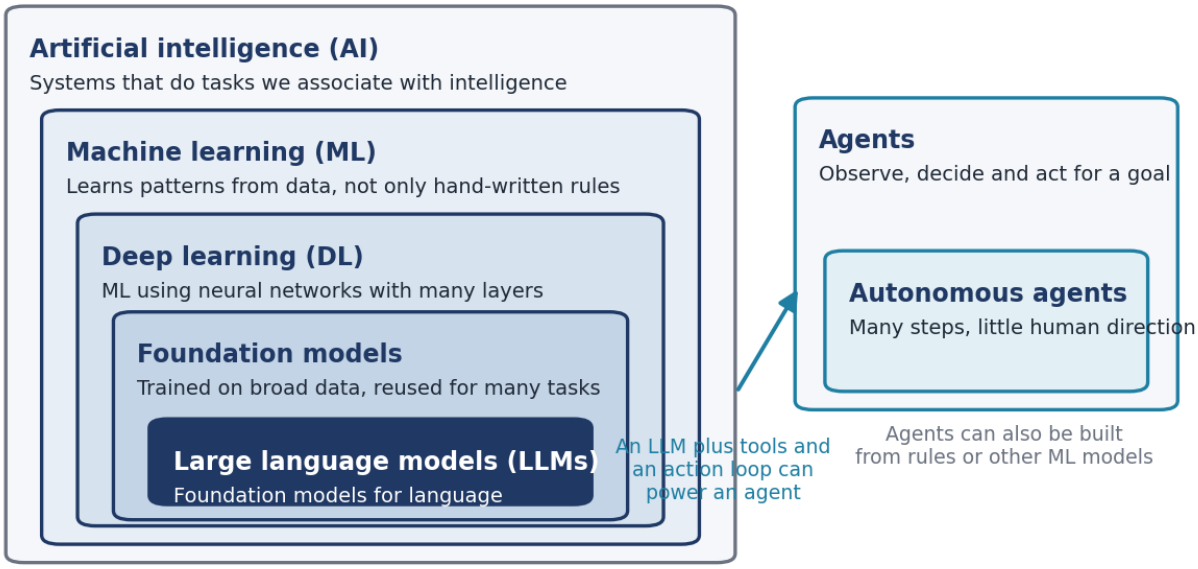


Figure 1. A simplified map of today's AI. The nested boxes on the left are the model families; the box on the right describes systems that act. Everything in this course lives in the "Machine learning" box.

- **Deep learning** is machine learning done with neural networks that have many layers; it is what made image recognition and speech recognition work well.
- **Foundation models** are very large deep-learning models trained on enormous, broad collections of data and then reused for many different tasks.
- **Large language models (LLMs)** such as the one behind a chat assistant are foundation models built for language.
- An **agent** is something different in kind: not a model but a *system* that observes a situation, decides what to do and takes actions towards a goal. An agent is often powered by an LLM combined with tools and a loop that lets it act, check and act again, but agents can also be built from plain rules or from other ML models.
- An **autonomous agent** is simply one that is trusted to take many such steps with little human direction.

You do not need to remember all of this now. The point is that these are different kinds of thing, that each layer is built on the one outside it, and that the layer everything grows from is the one we are about to open up: a program that learns from data.

2. A task we cannot write rules for

2.1 Some tasks have rules

Computers already help us with a huge range of tasks: recommending a film, keeping a car in its lane, tagging friends in photos, translating between languages, filtering spam, supporting a doctor's diagnosis. Before asking how a computer *learns*, it is worth noticing that many everyday tasks need no learning at all.

Ask yourself what instructions you would give a computer to work out the cost of three items at £4 each. You would say: multiply the quantity by the unit price. To convert 20°C to Fahrenheit, you would say: multiply by 1.8 and add 32. In both cases you can write down the rule, and once it is written the computer applies it perfectly, every time, to any input. A program like this is called **deterministic**: give it the same input and it produces the same output. Notice that deterministic means *repeatable*, not *correct*. A program with a typo in its conversion formula will faithfully give the same wrong answer every time.

2.2 Now try spam

Now try to write the rule for a different task on that list: deciding whether an incoming email is spam. A natural first attempt is "mark it as spam if it contains the word *free*". It takes one minute to see the trouble.

Email subject	What the rule does	What we wanted
Free lecture on Friday	Flags it as spam	A legitimate invitation gets blocked
Claim your cash reward	Lets it through	Obvious spam gets missed

You could patch the rule: add "reward", add "prize", make an exception for "lecture". But each patch invites the next. Language is endlessly varied, context changes what a word means, and the people sending spam actively reword their messages to slip past whatever you wrote last week. Rule-based spam filters do exist and can be useful, so the problem is not that rules are impossible. The problem is that *we* cannot sit down and write out a set of rules that works well across all the emails that will actually arrive. The real question is: **where should the decision rule come from, if not from us?**

2.3 Turn the problem around: learn from examples

Here is the move that defines machine learning. Instead of writing the rule, we show the computer a large number of past emails that people have already labelled as spam or not spam, and let a *learning algorithm* work out which combinations of clues (words, phrases, sender details, links) tend to go with each label. The result is a rule the computer wrote for itself, shaped by thousands of real cases rather than a few of our guesses. When a new email arrives, that learned rule reads its clues and makes a prediction.

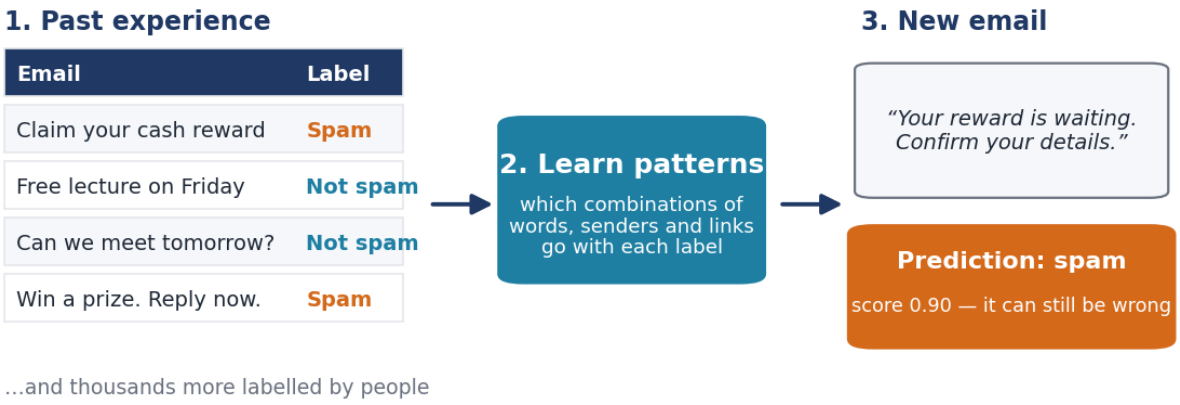


Figure 2. Learning from labelled emails. People supply past examples with their labels; a learning algorithm finds the patterns; the learned rule is then applied to new emails it has never seen. The subjects shown are invented teaching examples.

Look closely at the last step in Figure 2. The model reports that the new email is spam with a score of 0.90, and the caption adds that the prediction can still be wrong. Both halves matter. The model has not "understood" the email; it has noticed that its clues resemble clues that went with spam in the past. Legitimate and unwanted messages can share features, the available clues may not capture everything that matters, and a brand-new kind of message may look like nothing in the training examples. So a learned rule can give exactly the same, repeatable answer every time and still be uncertain or wrong. Repeatability and correctness, once again, are different things.

2.4 A definition we can now use

With the spam example in hand, the standard definition of machine learning reads naturally. Machine learning is about **computer programs that improve their performance at some task through experience**. Three words carry the weight, and each maps onto something concrete:

Element	In general	In the spam example
Task	The job we want done	Decide whether an incoming email is spam
Experience	The examples the program learns from, usually data	Past emails together with the labels people gave them
Performance	A measure of how well the job is done	How often the predictions are right on emails the program has not seen before

Whenever you meet a new machine-learning problem, in your own field or anyone else's, these three questions are the first thing to ask. What exactly is the task? What experience is available, and does it really contain the information needed? And how will we recognise good performance? If any of the three has no clear answer, the project is not ready, however exciting the technology.

Two words to read carefully

"Improve." It is tempting to think that a learning program gets better simply by seeing more data. It does not. More examples *can* support better learning, but only if they are relevant, correctly labelled and actually used in further training. Improvement is something to be checked with evidence, never assumed.

"Experience." A trained model does not keep learning every time it makes a prediction. Learning is a separate process that fits or updates the model from data; prediction simply applies the finished rule. A spam filter installed last year is exactly as clever today as it was then, unless someone has retrained it.

2.5 The real goal: generalisation

Suppose our learned spam filter classifies every one of the emails it was trained on correctly. Have we shown that it has learned something useful? Not yet. A rule that merely memorised those particular emails would also score perfectly on them, and would be useless on tomorrow's inbox. What we actually want is a rule that captures patterns which carry over to emails it has never seen. This property has a name, **generalisation**, and it is the central idea of the whole week. Nearly everything that follows, from how we split up data to how we judge whether a model is too simple or too complicated, exists to make sure we are measuring generalisation and not memorisation.

Pause and think

Think of a task in your own work or field where you can describe what a good decision looks like but would struggle to write down the rule for making it. What would the "experience" be? Who would have labelled it, and could those labels be wrong?

3. The experience: what data looks like

3.1 A dataset is a table

"Experience" sounds abstract, but in practice it is usually a table. Each **row** is one **example**: one email, one house, one customer. Each **column** records one property of that example, called a **feature**. And in the kind of learning we have been describing, one special column holds the thing we want to predict: the **target**, whose known values in the training examples are called **labels**. Here is the kind of table an estate agent might assemble to estimate house prices.

Floor area (m²)	Bedrooms	Sale price (£)
65	2	160,000
90	3	225,000
125	4	310,000
75	2	185,000

Blue columns are features (the inputs); the orange column is the label (the answer). Illustrative numbers only: four rows are enough to show the layout, nowhere near enough to learn from.

A useful way to picture it: each row is a quiz question whose answer we already know. The features are the question, the label is the answer, and training is the process of studying the answered

questions so that we can answer new ones. For a new house we supply the floor area and bedroom count and ask the model for the price; we do not, of course, supply the price. Notice too that labels do not have to be typed in by a person. A recorded sale price, or a record of whether a customer did or did not buy, is a perfectly good label that the world supplied for free.

3.2 Two kinds of experience: with and without answers

Not every table has an answer column. Imagine a retailer who records, for each customer, how many orders they placed last month and the average value of those orders. Nobody has written down which "type" of customer each one is; there is no label. We can still learn something. Plot the customers and two groups appear by themselves: occasional small spenders, and frequent large ones.

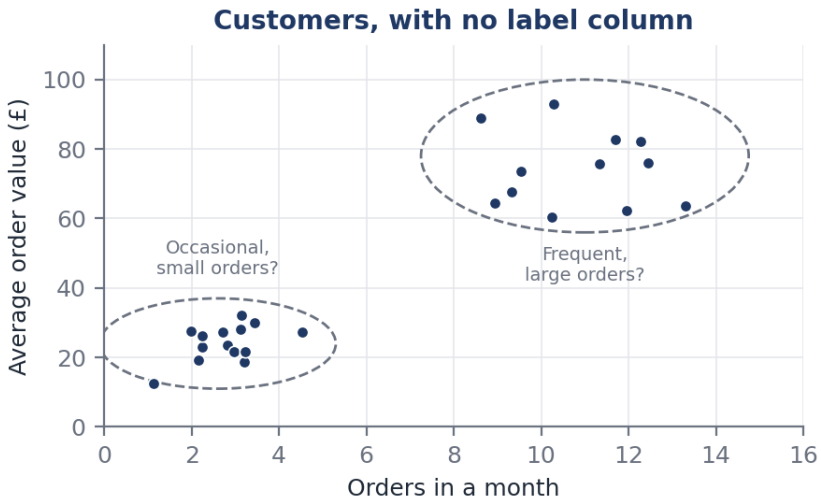


Figure 3. Customers described by two features and no label. A clustering method could find the two groups; it is up to people to decide whether the groups mean anything. Illustrative data.

This gives us the first big division in machine learning. **Supervised learning** uses examples that come with their answers (the house prices, the spam labels) and learns to predict the answer from the features. **Unsupervised learning** works with examples that have no answers and looks for structure in the features alone; grouping similar customers, called *clustering*, is the most familiar case. The word "unsupervised" does not mean "no humans involved": people still choose the question, the features and the method, and people must judge whether the groups the algorithm finds are useful. This course concentrates on supervised learning, which is where most real-world prediction happens.

3.3 Two kinds of answer: a category or a number

Within supervised learning there is one more split, and it depends only on what kind of thing the target is. Compare two tables.

Dataset A: will the customer buy?

Country	Age	Salary	Bought?
France	44	£72k	No
Spain	27	£48k	Yes

Country	Age	Salary	Bought?
Germany	30	£54k	No
France	48	£79k	Yes

Dataset B: what salary should we expect?

Years of experience	Salary
1.1	£39,300
1.5	£37,700
2.2	£39,900
3.2	£54,400

In Dataset A the answer is a **category**: yes or no. Predicting a category is called **classification**. Spam or not spam, cat or dog, disease or healthy, and problems with more than two categories such as which of ten digits is written on a cheque, are all classification. In Dataset B the answer is a **number**: a salary in pounds. Predicting a number is called **regression**. House prices, tomorrow's temperature and a patient's length of stay are regression targets.

The target decides, not the application. The same customer data could support a classification task (will they buy?) or a regression task (how much will they spend?), so you must say exactly which outcome you want. One small trap: if "did not buy" is coded as 0 and "bought" as 1, it is still classification. Those numbers are just names for categories; a prediction of 0.7 does not mean "bought seventy per cent". In regression, by contrast, the numbers are real quantities and the differences between them mean something.

> **A rule that saves embarrassment**
>
> Features must be things you will actually have **at the moment you need the prediction**. A model that predicts a house's sale price must not be given the sale price, or anything that quietly reveals it, as an input. That sounds obvious with houses, but it is one of the most common mistakes in practice: a column that is only filled in *after* the outcome is known sneaks into the features, the model looks brilliant in testing and useless in real life. Week 3 will come back to this.

4. What a model is, and what "fitting" means

We have been using the word *model* freely. Time to open it up. A **model** is a rule that turns features into a prediction: floor area and bedrooms in, price out; age and salary in, "will buy" or "will not buy" out. The rule is expressed in mathematics or code rather than English, but that is all it is. Learning a model happens in two steps, and separating them makes the whole idea much clearer.

1. **Choose a family of rules.** We decide what *shape* the rule is allowed to have: a straight line, say, or a smooth curve, or something far more flexible. This choice is made by a person.
2. **Let the data choose the member of the family.** Within that shape there are adjustable dials, called **parameters**. For a straight line they are its slope and where it crosses the axis. **Fitting**, or

training, is the process of turning those dials until the rule agrees with the training examples as well as it can. The procedure that turns the dials is the **learning algorithm**; the rule it ends up with is the **fitted model**.

4.1 Fitting a line: the salary example

The clearest picture of fitting is regression with a single feature. Suppose we have the salaries of twenty-four past employees together with their years of experience, and we choose the family of straight lines. Any line we draw makes a prediction for every employee; the vertical gap between a point and the line is that prediction's **error**.



Figure 4. Fitting a line to salary data. Left: a first guess, with each employee's error drawn as a vertical gap. Right: the fitted line, which makes those gaps as small as possible overall, and the prediction it gives for a new person with five years' experience. Illustrative data.

The line on the left is a poor fit: it is too flat, and the gaps are large. The learning algorithm's job is to find the slope and intercept that make the gaps smallest when all of them are taken together. The combined size of the gaps is called the **loss**, and fitting is a search for the parameters with the lowest loss. Some algorithms do this by repeated small adjustments (each nudge of the dials reduces the loss a little, and they keep going until no nudge helps), while for a straight line the answer can be calculated directly. The result is the line on the right, which rises by roughly £9,400 for every extra year of experience. Give it a new person with five years' experience and it reads off a prediction of about £71,000.

Two things to notice. First, even the fitted line does not pass through the points: experience alone does not determine salary, so some error is unavoidable, and the model is deliberately a *simplification*. Second, the loss used for fitting and the performance measure we report to people need not be the same thing; the loss is an internal quantity for the algorithm, and we will meet the reporting measures in Week 3.

4.2 Fitting a boundary: the customer example

For classification the same two steps apply, but the picture is different. Take Dataset A, keep just two features so that we can draw it, and plot every past customer by age and salary, colouring each point by whether they bought. A classifier's rule divides this picture into regions: land in one region and the prediction is "buy", land in the other and it is "no buy". The line or curve that separates the

regions is called the **decision boundary**, and fitting means choosing its position and shape from the labelled examples.

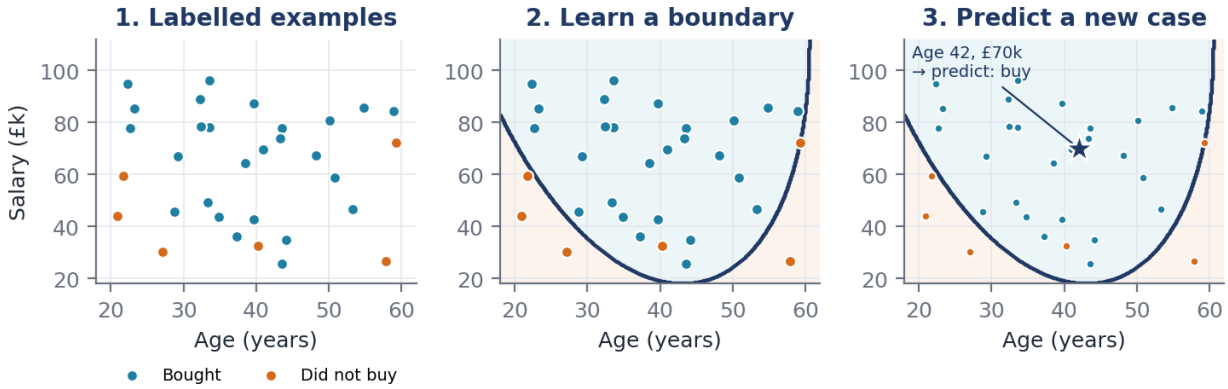


Figure 5. Learning a classifier. (1) Past customers plotted by age and salary, coloured by the known outcome. (2) A boundary fitted to those examples; the shaded regions are the model's predictions everywhere. (3) A new customer, age 42 earning £70k, falls on the "buy" side. Thirty-two invented customers.

In this illustration the people who bought cluster in a U-shaped region (younger and older customers on higher salaries), so the fitted boundary is a curve. A new customer aged 42 earning £70,000 lands inside the curve and is predicted to buy. As with the spam filter, that prediction may be wrong: age and salary do not decide every purchase, and the boundary was shaped by only thirty-two past cases. With two features the boundary is a line we can see; with twenty features it becomes a surface in a space we cannot draw, but the idea is identical. A model that also outputs a *score* (how far inside the region the customer sits, roughly) carries more information than the boundary alone, and Week 2 will use exactly that.

> **Learning and predicting are different moments**
>
> During **training**, the data shape the model: the dials move. Once training is finished, the model is fixed. **Prediction** (also called *inference*) just applies the fixed rule to a new case, and does not change the model. If you want the model to learn from new cases, you have to run the learning process again with them included.

5. How do we know whether it works?

5.1 The exam analogy

Here is the question the whole course keeps coming back to. We have a fitted model. It does well on the examples it was trained on. Can we trust it on new cases?

Think about how we judge a student. If a student is given the past exam papers *with* the answers and then tested on exactly those papers, a perfect score tells us almost nothing: they may have understood the subject, or they may have memorised the answers. The only way to tell the difference is to set questions they have not seen. Machine learning works the same way. Mistakes on the training examples are the **training error**; mistakes on fresh examples the model never saw during training are the **test error**. Training error tells us how well the model fits the past; only test error tells us how well it generalises, which is what we actually care about.

That is why, before any training happens, we set some of our labelled examples aside and lock them away. The model is fitted on the rest, the **training set**. Only at the end do we unlock the **test set**, let the model predict each case, and compare its predictions with the labels we have been keeping secret.

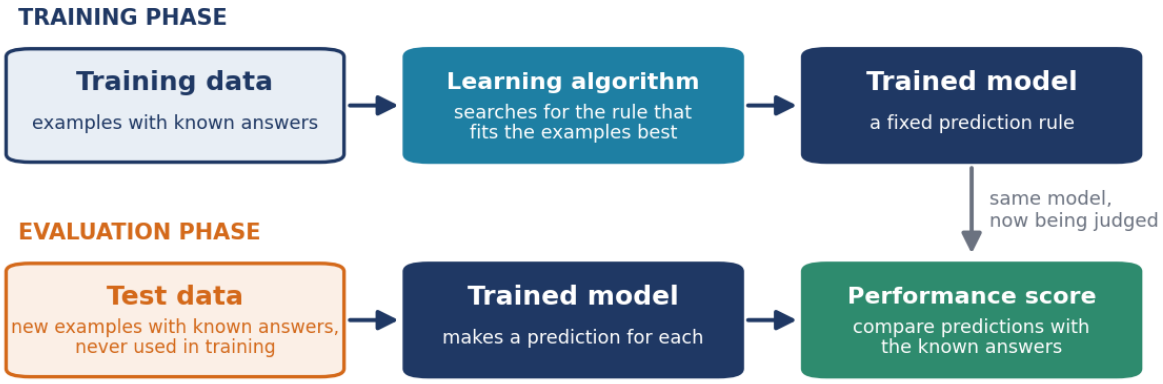


Figure 6. The full supervised-learning pipeline. Training uses one portion of the labelled data to fit the model; evaluation applies the finished model to a separate, unseen portion and scores its predictions against the known answers.

5.2 Too simple, too flexible, about right

Now we can return to the first step of learning, the choice of family, and see why it matters so much. Below, three different families have been fitted to the *same* thirty-two customers from Figure 5. Twelve further customers, drawn as crosses, were kept aside as a test set.

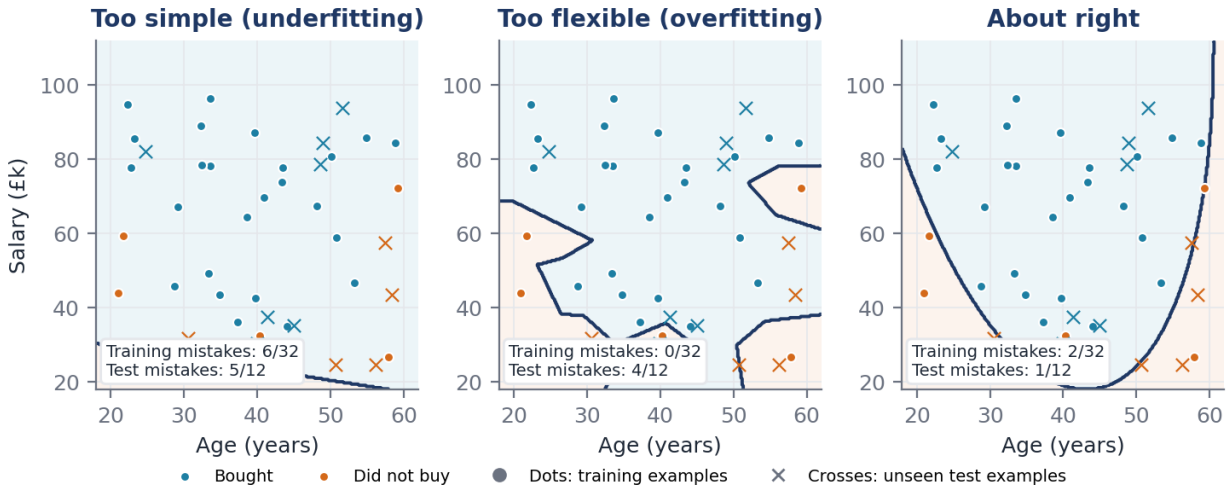


Figure 7. Three models fitted to the same training customers (dots) and then checked on the same unseen customers (crosses). The counts of mistakes are printed in each panel. Synthetic illustration.

Family of rule	Training mistakes	Test mistakes	What is going on
Straight boundary	6 of 32	5 of 12	Underfitting. Too rigid to follow the U-shape, so it does poorly even on the data it learned from.

Family of rule	Training mistakes	Test mistakes	What is going on
Very flexible boundary	0 of 32	4 of 12	Overfitting. Bends around every single training point, including the odd ones, and that detail does not carry over.
Smooth curved boundary	2 of 32	1 of 12	About right. Captures the main pattern, accepts a couple of training mistakes, and generalises best.

Three lessons sit in that table. **Underfitting** is a model that is too simple to capture the real structure: it does badly everywhere. **Overfitting** is a model that is so flexible it fits the noise and quirks of its particular training sample: it looks perfect on training data and disappoints on new data. And the most important lesson is the one that surprises people: **zero training mistakes is not the goal**. The very flexible model achieved it and was the worse model. The middle option made two mistakes on its own training data and was the best. Training error on its own simply cannot tell you which model to trust.

5.3 Choosing flexibility with evidence

If we imagine sliding smoothly from very simple families to very flexible ones, the pattern in Figure 7 becomes a curve.

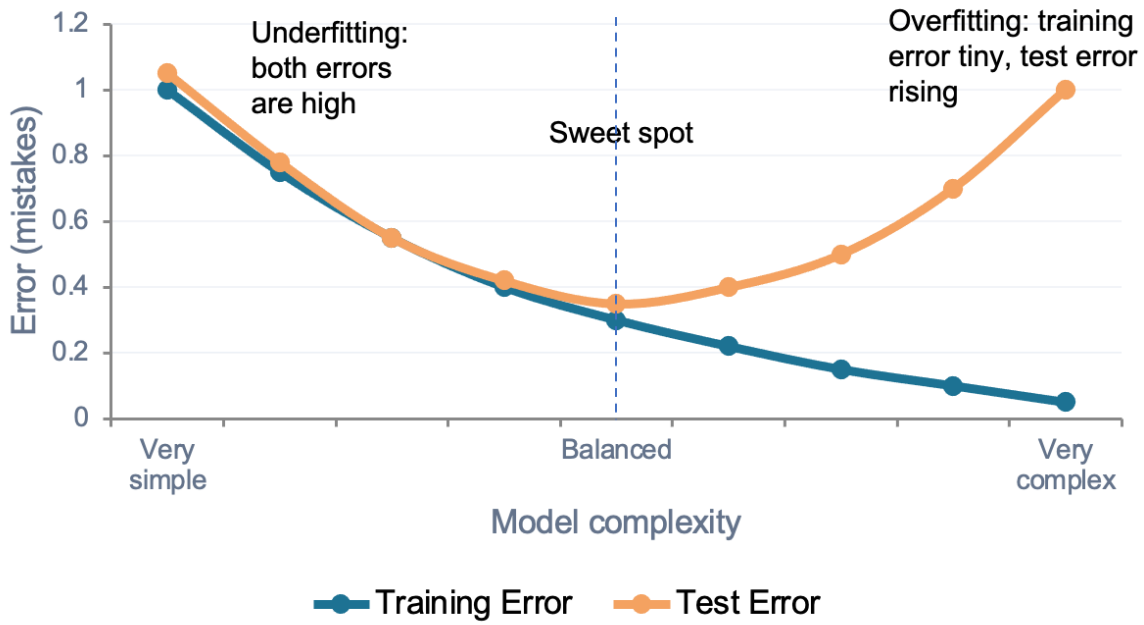


Figure 8. As a model becomes more flexible, training error keeps falling, but test error falls and then rises again. The best model sits in the dip, not at the right-hand end. A common pattern, though not a universal law.

Training error falls the whole way: a more flexible model can always fit the past more closely. Test error falls at first, because the model is capturing real structure, and then rises, because it has started memorising noise. The model to choose is the one at the bottom of the dip. The curve is a typical pattern rather than a law, and for a simple relationship the best model really may be a

straight line; the point is that the decision is made by looking at performance on **unseen** data, not by picking the model that looks cleverest.

> **Bias and variance, in plain words**
>
> You will meet these two words whenever this trade-off is discussed. **Bias** is systematic error: a model too simple to capture the pattern will be wrong in the same direction no matter which training sample it sees. **Variance** is sensitivity to the sample: a very flexible model would come out quite different if we trained it on a different thirty-two customers, because it chases their particular quirks. Underfitting is high bias; overfitting is high variance; the sweet spot balances them. Note that this technical sense of "bias" is unrelated to social or demographic bias, which is a separate and equally important topic.

5.4 Evaluating fairly, and a warning about accuracy

A test set only tells the truth if it resembles real use. If the model will be used on future months, test it on later data than it was trained on. If several records belong to the same person, keep them together on one side of the split, or the model gets to "see" people it will be tested on. If you rescale features or fill in missing values, work out how to do so from the training data alone. Each of these is a way of making sure the exam really is unseen.

Finally, a question we have carefully avoided: *which* score? The natural choice is **accuracy**, the fraction of predictions that are correct, and it is where we will begin in Week 3. But a single number can hide a great deal. Suppose only one email in a hundred is spam; a filter that calls everything "not spam" is 99 per cent accurate and completely useless. Missing a spam message and hiding a legitimate one are different mistakes with different costs, and a responsible evaluation has to look at them separately. That is where the course is heading.

6. Pulling it together

The whole week fits into three questions asked of three examples.

	Spam filter	House price / salary	Will the customer buy?
Task	Classification: spam or not	Regression: a price or salary in pounds	Classification: yes or no
Experience	Past emails labelled by people	Past sales or employees with their recorded prices or salaries	Past customers with their recorded outcome
Model	A rule over words, senders and links	A line (or curve) through the points	A boundary dividing the age–salary picture
Performance	Mistakes on emails never seen in training	Size of the errors on unseen houses or people	Mistakes on unseen customers

Key terms

Term	Meaning
Example	One row of the data: one email, one house, one customer.

Term	Meaning
Feature	A property of an example used as input to the model.
Target / label	The thing we want to predict; its known value in a training example is the label.
Supervised learning	Learning from examples that come with their answers.
Unsupervised learning	Finding structure in examples that have no answers, for instance clustering.
Classification	Supervised learning where the target is a category.
Regression	Supervised learning where the target is a number.
Model	A rule that maps features to a prediction; a chosen family plus fitted parameters.
Fitting / training	Adjusting the parameters so the model agrees with the training examples as well as possible.
Loss	The quantity the learning algorithm tries to make small during fitting.
Decision boundary	The line or surface where a classifier's prediction changes from one category to another.
Generalisation	Performing well on examples that were not used for training. The goal.
Training / test error	Mistakes on the training examples / on held-out, unseen examples.
Underfitting / overfitting	Too simple to capture the pattern / so flexible it fits the noise of the training sample.

Check yourself before the webinar

Answer each in your own words first, then compare with the pointer underneath.

1. In the house table, which columns are features and which is the label? What would change if the estate agent instead wanted to group similar houses without using prices?

Floor area and bedrooms are features; sale price is the label. Grouping without prices would be unsupervised learning (clustering); the price column would simply not be used as a target.

2. "Did not buy" is coded 0 and "bought" is coded 1. Is predicting this column classification or regression? Why?

Classification. The numbers are names for categories; the gap between 0 and 1 is not a quantity that means anything.

3. A colleague says their model is excellent because it makes zero mistakes on the training data. What would you ask them?

How it does on data it was not trained on. Zero training mistakes is consistent with a model that has memorised the sample (overfitting) and may generalise badly.

4. Why do we keep a validation set as well as a test set?

Choosing between models using the test set lets the test set influence the design, so its final score is no longer an honest estimate. Validation data absorb those choices; the test set is looked at once, at the end.

5. The spam filter says “0.90” for an email. Does that mean it is 90 per cent certain, and does it mean the prediction is right?

Neither automatically. A score of 0.90 is meaningful only if the model is well calibrated, meaning that among many emails scored around 0.90, roughly nine in ten really are spam. And a high score is still a prediction about a group of similar cases, not a guarantee about this one.

> **Next week**
>
> We take one real classification problem and work it through from start to finish with a specific model, **logistic regression**. Despite its name it is a classifier, and it produces exactly the kind of score and decision boundary we met in Figure 5. We will ask why the problem suits machine learning, how the model turns features into a class labels, how we fit it, and how we test it on data it has not seen.

Sources and reuse

This reading was written for the DAIM short course and is intended for open sharing and reuse. All numerical tables, the customer, salary and house figures and the email subjects are invented teaching illustrations, generated for this document; they are not real datasets, and the mistake counts in Section 5 describe those invented examples only. The explanations draw on the following publicly accessible educational sources, checked 1 October 2026.

[1] Google, Introduction to Machine Learning: What is Machine Learning?

[2] Google, Machine Learning Glossary

[3] Paul E. Black, NIST Dictionary of Algorithms and Data Structures: deterministic algorithm

[4] Google, Framing an ML problem

[5] Google, Introduction to Machine Learning: Supervised Learning

[6] Tom Mitchell, Machine Learning (McGraw-Hill, 1997); the task / experience / performance definition

[7] scikit-learn developers, Probability calibration

[8] Google, Machine Learning Crash Course: Overfitting and generalization (overfitting, model complexity, dividing datasets)

[9] Google, Machine Learning Crash Course: Linear regression (loss, gradient descent)

[10] scikit-learn developers, Single estimator versus bagging: bias–variance decomposition

[11] Google, Machine Learning Crash Course: Classification: accuracy, recall, precision and related metrics

[12] scikit-learn developers, Common pitfalls and recommended practices

[13] scikit-learn developers, Cross-validation: evaluating estimator performance

[14] Google, Machine Learning Crash Course: Logistic regression: calculating a probability

[15] Bommasani et al., On the Opportunities and Risks of Foundation Models (2021)

[16] Anthropic, Building effective agents (2024)

Google developer text is licensed under Creative Commons Attribution 4.0; this reading paraphrases those explanations and adds its own examples. scikit-learn documentation carries a BSD licence. NIST publications are generally public domain in the United States. Publicly accessible and public domain are not the same thing: the Google and scikit-learn materials retain their stated licences.